

MODULE 24 · DOMAIN 3 - CONTINUOUS IMPROVEMENT FOR EXISTING SOLUTIONS

Advanced Monitoring

Tracing, canaries, and when the managed open-source stack is the answer.

DOMAIN 3 of 4 EXAM WEIGHT 25% MODULE 24 of 44

Metrics tell you **something** is slow. A trace tells you **which of your twelve services** is slow.

AWS X-Ray

- ◆ A **trace** is one request's journey. **Segments** are the work each service did, and **subsegments** break that down further.
- ◆ The **service map** is drawn from traces automatically, showing latency and error rates on every edge.
- ◆ **Sampling rules** control how much you record. The default records a first request each second, then a small percentage.
- ◆ **Annotations are indexed** and searchable. Metadata is stored and not searchable. That distinction gets tested.

ON THE EXAM

Sampling is why X-Ray stays affordable at scale. You trace enough requests to see the pattern, not every request.

The instrumentation shift

- ◆ **The X-Ray SDKs are in maintenance.** AWS now points new instrumentation at OpenTelemetry.
- ◆ **ADOT** is AWS's supported OpenTelemetry distribution. Instrument once, then choose the destination in collector config.
- ◆ One instrumentation can feed **CloudWatch, X-Ray, Managed Prometheus, OpenSearch** and third-party tools at the same time.
- ◆ Managed Lambda layers auto-instrument a function with **no code change**. EKS uses an add-on; ECS and EC2 run the collector.

ON THE EXAM

X-Ray the service is fine and fully supported. It is the SDKs that are in maintenance, and that distinction matters.

CloudWatch Synthetics

- ◆ **A canary is a scripted request run on a schedule**, from outside, whether or not any real user is trying.
- ◆ Blueprints cover heartbeat checks, API workflows, broken link checking, and visual comparison against a baseline.
- ◆ Canaries catch what internal health checks miss: expired certificates, DNS faults, a broken CDN, a bad third party.
- ◆ They also cover the quiet hours, when no real traffic would reveal a problem.

ON THE EXAM

A load balancer health check proves the target is up. A canary proves a user in another country can complete a purchase.

Synthetic or real user monitoring?

SYNTHETICS CANARIES

A robot, on a schedule

- Runs with no real traffic
- Covers nights and quiet periods
- Consistent, comparable results
- Only tests what you scripted

CLOUDWATCH RUM

Actual users, actual browsers

- Real devices, networks and geographies
- Page load and rendering performance
- Session replay for UX faults
- Nothing to see when nobody visits

THE TIEBREAKER

Detecting an outage before customers do means canaries. Explaining why some customers are slow means RUM.

Containers and the open-source stack

- ◆ **Container Insights** gives cluster, node, pod and container metrics for ECS and EKS, with a curated console view.
- ◆ **Amazon Managed Service for Prometheus** is a managed Prometheus back end. You keep PromQL, alert rules and recording rules.
- ◆ **Amazon Managed Grafana** visualises it, alongside CloudWatch, OpenSearch, Timestream and third-party sources.
- ◆ Neither is cheaper than CloudWatch. You choose them to keep existing dashboards, queries and skills.

ON THE EXAM

Existing PromQL alert rules or Grafana dashboards is the whole signal. Without those, CloudWatch is the simpler answer.

Where the money goes

- ◆ **Tracing everything is the expensive mistake.** Sampling rules exist so you can trace representatively.
- ◆ Canaries are charged per run, so a one-minute schedule on forty endpoints costs real money. Match frequency to the SLO.
- ◆ Container Insights ingests a great many metrics. Scope it to the clusters that matter rather than everything.
- ◆ Managed Prometheus charges per sample ingested and stored, so cardinality is the cost driver.

ON THE EXAM

High-cardinality labels are the Prometheus cost trap. A label containing a user ID multiplies series without limit.

Reading the scenario

THE SCENARIO SAYS

Which of many services is causing the latency

Customers noticed the outage before we did

Some users are slow, others are fine

Existing PromQL rules and Grafana dashboards

Tracing cost has become significant

Instrument once, send to several backends

IT IS POINTING AT

X-Ray distributed tracing

Synthetics canaries

CloudWatch RUM

Managed Prometheus and Managed Grafana

Sampling rules

ADOT

ON THE EXAM

Ask whether the gap is where, who, or how much. Tracing answers where, RUM answers who, sampling answers how much.

Meridian Freight's booking API has a p99 latency of 4 seconds against a 500 millisecond target. The request passes through an API Gateway, six microservices on ECS, two databases and a third-party address validation service. Each team's own dashboards show acceptable latency.

Incident calls have twice ended without identifying a cause.

What should the solutions architect implement?

- ☐ A Distributed tracing with X-Ray, instrumented through ADOT, and review the service map
- ☐ B Detailed monitoring on every ECS task
- ☐ C A CloudWatch dashboard combining all six services' latency metrics
- ☐ D Increase log verbosity across all six services

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A X-Ray tracing via ADOT

A trace follows one slow request through every hop and attributes the time, including the third party. The service map shows it directly.

C A combined dashboard

The strong near miss. Six averages on one screen still cannot connect the slow requests to each other.

B Detailed monitoring

Higher resolution on the same per-service averages. Every team already sees acceptable latency at five-minute granularity.

D More logging

Produces six streams with no shared request identifier, so correlating a single journey remains manual and slow.

THE SIGNAL

Every team's dashboard looking fine is the giveaway. Averages hide a tail; only a trace follows one actual request.

A retailer's checkout was unavailable for 90 minutes overnight. Every internal health check passed throughout: instances healthy, target groups healthy, database responsive. The cause was an expired TLS certificate on a third-party payment widget the checkout page loads.

Customers reported it in the morning.

What should the solutions architect implement?

- ☐ A CloudWatch Synthetics canary that completes a checkout flow every five minutes
- ☐ More aggressive ALB health check thresholds
- ☐ CloudWatch RUM to capture real user experience
- ☐ An X-Ray trace on the checkout service

◆ PAUSE AND DECIDE

What should the solutions architect implement?

- A

A checkout-flow canary

Exercises the whole journey from outside, including the third-party widget, and runs overnight when no real traffic would reveal it.
- B

Tighter health checks

Health checks probe your targets, which were genuinely healthy. No threshold detects a broken third-party asset.
- C

CloudWatch RUM

The strong near miss. RUM would have shown it, and only once a real user visited. Overnight there were none.
- D

X-Ray tracing

Traces server-side request paths. A certificate failing in the browser on a third-party asset never reaches your services.

THE SIGNAL

Overnight is the planted word. It eliminates RUM, because real user monitoring needs real users.

A company migrating 200 microservices from on-premises Kubernetes to EKS has four years of Prometheus recording rules, alerting rules and roughly 300 Grafana dashboards. The SRE team is fluent in PromQL. The migration must complete in six months.

A consultant has proposed rewriting everything as CloudWatch dashboards and alarms.

What should the solutions architect recommend?

- ☐ A Rewrite as CloudWatch dashboards and alarms, as proposed
- ☐ B Amazon Managed Service for Prometheus and Amazon Managed Grafana, with ADOT collecting metrics
- ☐ C Self-manage Prometheus and Grafana on EKS
- ☐ D Container Insights only

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A** Rewrite for CloudWatch
Four years of rules and 300 dashboards rebuilt inside a six-month migration, discarding the team's PromQL fluency.
- B** Managed Prometheus and Grafana
Rules and dashboards keep working, PromQL skills transfer, and AWS runs the back end instead of the team.
- C** Self-manage both
Preserves compatibility and keeps the operational burden the migration was meant to reduce, including scaling Prometheus itself.
- D** Container Insights only
Gives infrastructure metrics and none of the application recording rules or dashboards. It replaces a fraction of what exists.

THE SIGNAL

Pair this with module fifteen's ECS versus EKS pair. Existing artefacts and skills change the answer, and the reasoning is identical.

X-Ray costs have reached \$19,000 a month on a platform serving 40,000 requests per second. Tracing was enabled with a rule recording 100% of requests, because the team wanted complete visibility. They still mostly look at the service map and p99 latency. They do occasionally need to find a specific customer's failed request.

What should the solutions architect recommend?

- ☐ A Disable X-Ray and rely on CloudWatch metrics
- ☐ B Tune sampling rules: a low base rate, with a higher rate for errors and specific paths
- ☐ C Reduce the X-Ray trace retention period
- ☐ D Move tracing to a self-managed Jaeger deployment

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Disable X-Ray

Removes the cost and the capability. Scenario one showed what happens to a distributed system with no tracing.

C Shorter retention

X-Ray cost is dominated by traces recorded and scanned, not by how long they are kept.

B Tune sampling rules

A service map and p99 need a representative sample, not every request. Higher sampling on errors preserves the failed-request use case.

D Self-managed Jaeger

Trades a service bill for cluster, storage and operations work, plus re-instrumentation. Rarely cheaper once staff time counts.

THE SIGNAL

The team's actual usage is stated: service map and p99. Both are statistical, and neither needs a complete record.

Canaries pass consistently and server-side latency is well within target, yet support tickets report that the site is slow. The complaints cluster among users in Brazil and India, on mobile devices, and mention pages rendering incompletely.

The team cannot reproduce it from the office.

What should the solutions architect implement?

- ☐ A CloudWatch RUM, with session replay to see what those users experience
- ☐ B More canaries, running from additional Regions
- ☐ C X-Ray tracing on the backend services
- ☐ D Increase the ALB idle timeout

◆ PAUSE AND DECIDE

What should the solutions architect implement?

- A** RUM with session replay
Captures real devices, networks and geographies, and replay shows pages rendering incompletely. Exactly what nobody can reproduce.
- B** Canaries from more Regions
The strong near miss. It adds geography, and a canary still runs a scripted headless browser on a data-centre connection.
- C** X-Ray tracing
Server-side latency is already confirmed as within target. The problem lives in the browser.
- D** Longer idle timeout
A server-side connection setting for a client-side rendering symptom. Nothing suggests connections are being dropped.

THE SIGNAL

Pair this with scenario two. There, overnight ruled RUM out. Here, real devices on real networks rules canaries out.

A company must send traces to X-Ray for AWS-native troubleshooting, metrics to Amazon Managed Service for Prometheus for the SRE team's alerting, and a copy of both to a third-party platform the security team has already licensed.

The applications are being instrumented now, and the team does not want to instrument three times.

What should the solutions architect recommend?

- A Instrument with the X-Ray SDK and build exporters for the other two destinations
- B Instrument with ADOT, and configure the collector to export to all three destinations
- C Instrument three times, once per destination's native SDK
- D Send everything to CloudWatch and use subscription filters to forward it

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 X-Ray SDK plus custom exporters
The X-Ray SDKs are in maintenance, and this means writing and maintaining exporters that ADOT already ships.
- B

 ADOT to all three
Instrument once with OpenTelemetry, then choose destinations in collector configuration. Adding a fourth is a config change.
- C

 Instrument three times
Triples the instrumentation work and the risk of the three views disagreeing about the same request.
- D

 CloudWatch plus subscriptions
Works for logs and does not carry traces or Prometheus metrics to those back ends in a usable form.

THE SIGNAL

Instrument once, choose destinations in configuration. That property is the whole argument for OpenTelemetry.

What to carry out of this module

- ◆ **Metrics say something is slow; a trace says which hop.** Healthy per-service dashboards hide a p99 problem.
- ◆ **Canaries run without users; RUM needs users.** Overnight detection means canaries. Explaining slow users means RUM.
- ◆ **Managed Prometheus and Grafana exist to preserve rules, dashboards and skills,** not to be cheaper.
- ◆ **Sampling is not a compromise.** Service maps and percentiles are statistical and need a representative sample.

ON THE EXAM

ADOT is now the instrumentation layer. The X-Ray SDKs are in maintenance, while X-Ray the service remains fully supported.